

Applied Computer Vision and Deep Learning

UBCO Master of Data Science – DATA 589

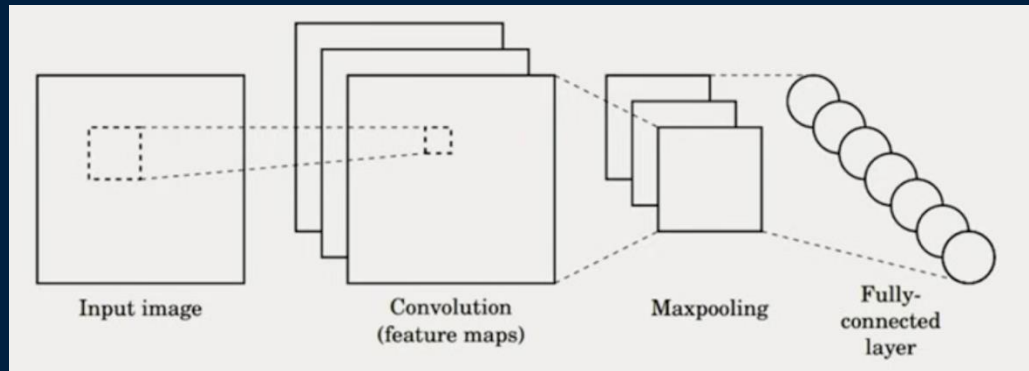


CNN Architecture

CNN architecture for Classification

1. Convolution: apply filters to generate feature maps
2. Non-linearity: often ReLU
3. Pooling: downsampling operation on each feature map

```
import torch
import torch.nn as nn
cnn_block = nn.Sequential(
    nn.Conv2d(1, 8, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2))
```



Convolutional Layers

- Each neuron looks only at a **local patch** of the input image
- The neuron computes a **weighted sum + bias** over that patch
- The same set of weights is **reused across all spatial locations**
- This creates a **feature map** showing where a pattern appears
- Different filters learn **different visual patterns**
- **Non-linear activation** allows detection of **complex structures**

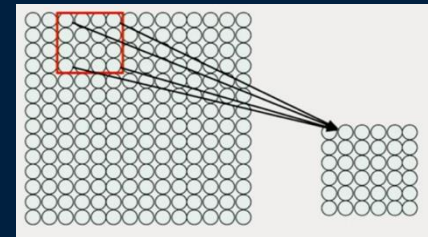
Key takeaway:

A convolutional layer detects *what* is present and *where* it appears in the image.

Local Connectivity

Convolution Operation (Single Output Neuron): For a neuron at spatial location $((p, q))$ in the hidden layer/feature map:

- Take input from patch
- Compute weights
- Apply bias



The bias shifts the output value, allowing the filter to activate even when the weighted sum is zero.

The weights decide *what* pattern we look for; the bias decides *how strong the evidence must be* before we respond.

$$y_{p,q} = \sum_{i=1}^k \sum_{j=1}^k w_{i,j} x_{p+i, q+j} + b$$

Where:

- x = input image (or previous feature map)
- $w_{i,j}$ = filter (kernel) weights
- $k \times k$ = filter size (e.g., 3×3 , 4×4)
- b = bias term
- $y_{p,q}$ = output value at position (p, q)

CNNs

Feature map layer dimensions:

$d \times h \times w$

- **Height (h)** – spatial height of the feature map
- **Width (w)** – spatial width of the feature map
- **Depth (d)** – number of filters (number of feature maps)

Each filter produces one feature map and stacking them forms the depth.

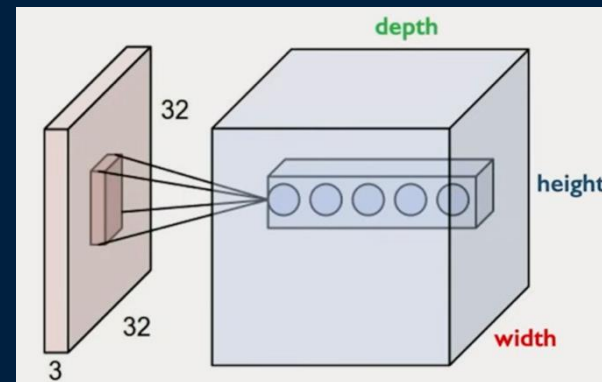
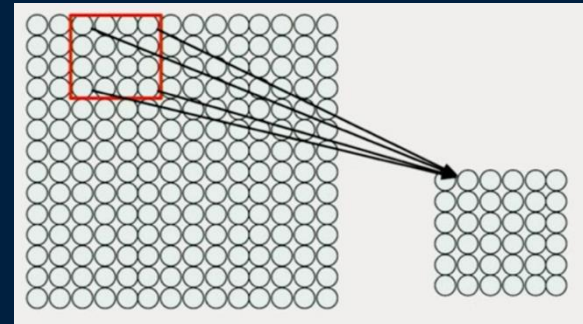
Stride

Stride controls how far the filter moves at each step

- **Larger stride** → **smaller output spatial resolution**
- **Smaller stride** → **finer spatial detail**

Receptive Field

- The receptive field is the region of the input image that affects one output neuron
- Deeper layers have **larger effective receptive fields**
- Enables learning from local patterns → global structure



```
torch.nn.Conv2d( in_channels=3, out_channels=d, kernel_size=(h,w), stride=s )
```


Padding

Padding is primarily used to prevent the spatial dimensions of the feature maps from shrinking after each convolution layer, allowing for deeper architectures without losing information at the edges. By adding a border of zeros, padding ensures that the pixels at the corners and edges of the image are processed as frequently as those in the center, preserving critical spatial features throughout the network.

No Padding (5x5)					With Padding p=1 (7x7)						
x	x	x	x	x	0	0	0	0	0	0	0
x	x	x	x	x	0	x	x	x	x	x	0
x	x	x	x	x	0	x	x	x	x	x	0
x	x	x	x	x	0	x	x	x	x	x	0
x	x	x	x	x	0	x	x	x	x	x	0
x	x	x	x	x	0	x	x	x	x	x	0
x	x	x	x	x	0	0	0	0	0	0	0

```
conv = nn.Conv2d(in_channels=3, out_channels=D, kernel_size=(H, W), stride=S, padding=P)
```

CNNs

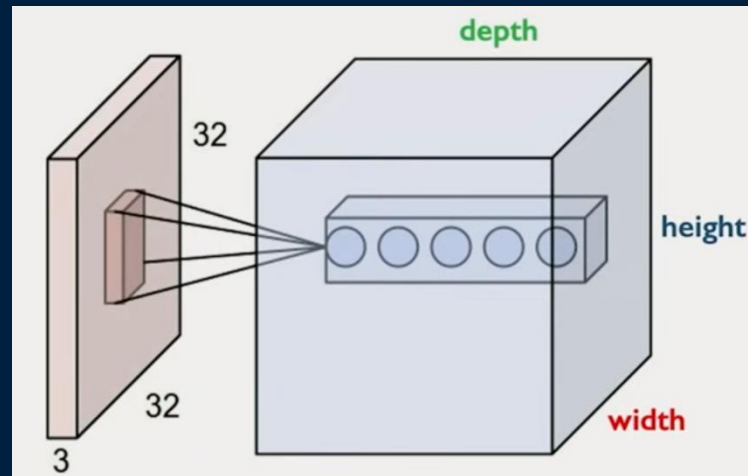
Visual mental model

Filter → 3D block (channels × height × width)

Feature map → 2D image (height × width)

Output of a conv layer → stack of 2D feature maps

- One filter gives one feature map.
- More filters give more feature maps.
- Filter depth matches input depth.



Feature map height and width:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - H}{S} \right\rfloor + 1$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2P - W}{S} \right\rfloor + 1$$

- H_{in} : height of the input image
- W_{in} : width of the input image
- H : height of the filter/kernel
- W : width of the filter/kernel
- P : padding
- S : stride

```
conv = nn.Conv2d(in_channels=3, out_channels=D, kernel_size=(H, W), stride=S, padding=P)
```

Question

Question: If we have a 32×32 RGB image and apply 64 filters of size 3×3 , which statement is correct?

- A. Each filter has size $3 \times 3 \times 1$, and the output has 3 feature maps
- B. Each filter has size $3 \times 3 \times 3$, and the output has 64 feature maps
- C. Each filter has size $32 \times 32 \times 3$, and the output has 1 feature map
- D. Each filter has size $64 \times 64 \times 3$, and the output has 64 feature maps
- E. Each filter has size $3 \times 3 \times 64$, and the output has 3 feature maps

Question

Question: What is the purpose of **padding** in convolutional layers?

- A. Increase the number of filters
- B. Prevent feature maps from shrinking
- C. preserve edge information
- D. Reduce dataset size
- E. B and C

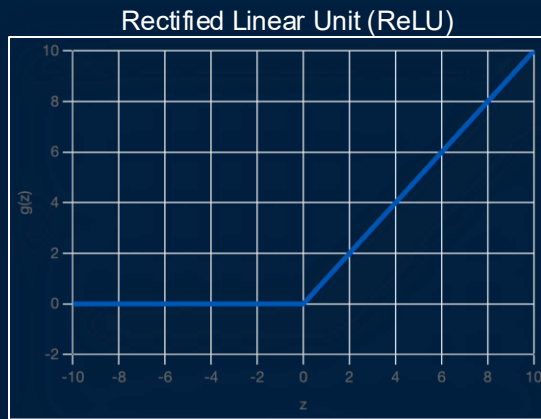
Non-Linearity

Why **Non-Linearity** Is Needed

- Without non-linear functions, stacking convolution layers results in a single linear operation
- Non-linearity allows neural networks to model complex, non-linear relationships
- Essential for learning meaningful visual patterns

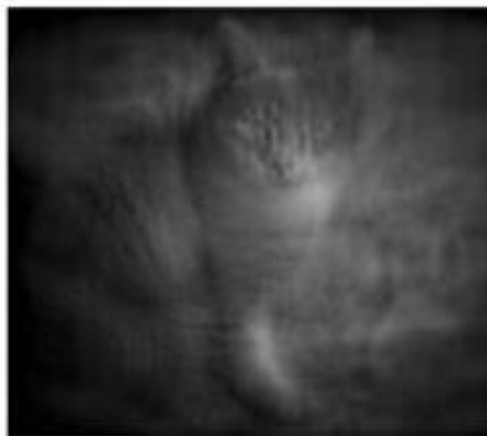
A common non-linear function: Rectified Linear Unit (ReLU)

- Applied **after each convolutional layer**
- Operates **pixel-by-pixel** on feature maps
- Replaces all negative values with zero
- Keeps positive values unchanged



$$g(z) = \max(0, z)$$

Without Activation



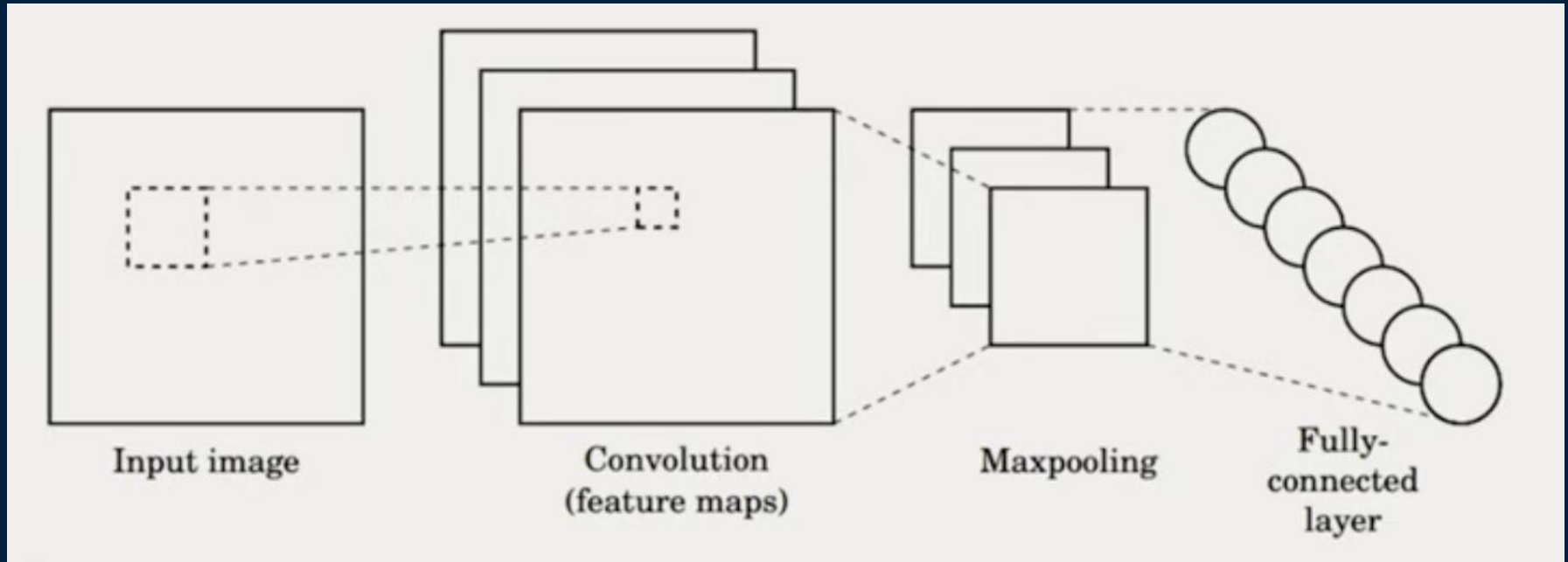
negative & positive values

With ReLU Activation



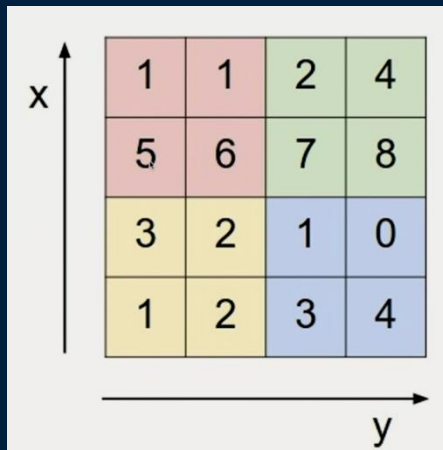
only non-negative values

CNN – Architecture : Pooling



Pooling

- Pooling reduces the **spatial resolution** of feature maps
- Operates **independently on each feature map**
- No learnable parameters
- Improves computational efficiency and robustness



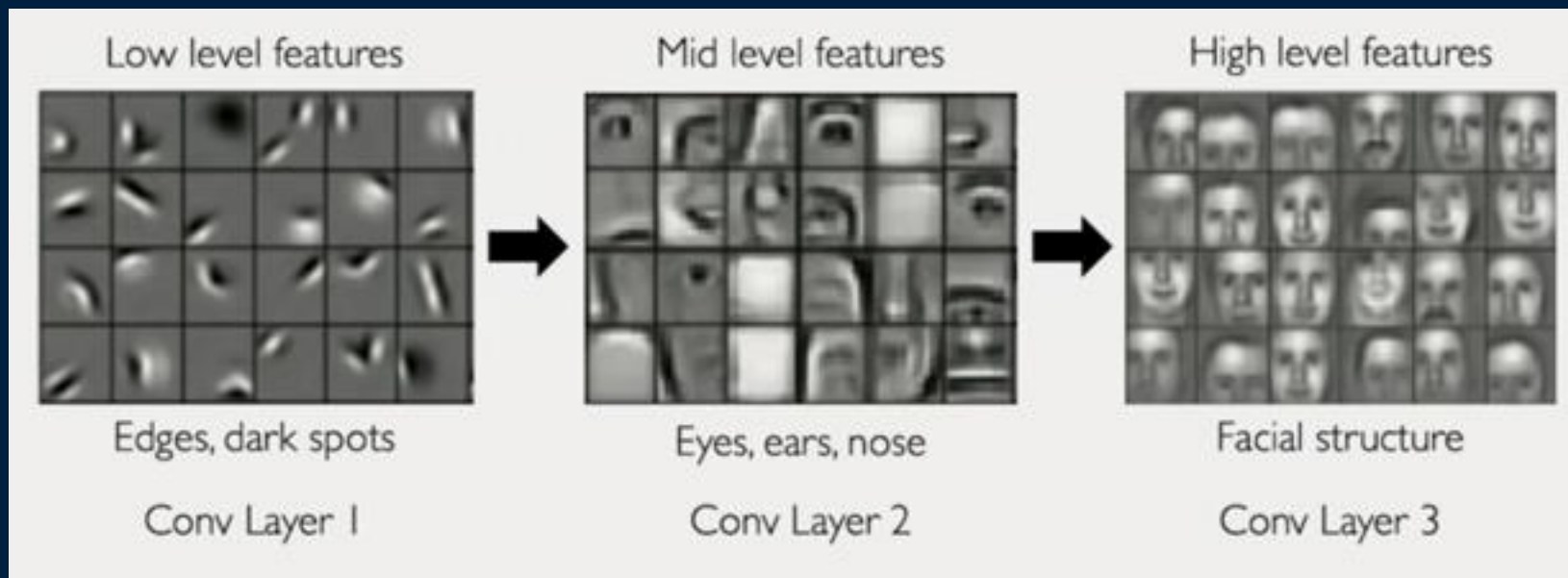
Max Pooling
with 2x2 filters
and stride 2



Pooling preserves spatial invariance by summarizing local regions, making the network robust to small spatial shifts of features. Most common pooling operations are max and average.

In most CNN models Pooling layers do not have learnable parameters; the pool size and stride are fixed hyperparameters chosen during model design.

CNN Layers



Question

Question: Which of the following statements about **Convolutional Neural Networks (CNNs)** is **FALSE**?

- A. A convolution filter is applied to local regions of an image to extract features such as edges or textures.
- B. Weight sharing in CNNs reduces the number of parameters and allows the same feature to be detected at different spatial locations.
- C. Pooling layers usually have learnable parameters that are updated during training.
- D. A feature map represents the response of a filter applied across the spatial dimensions of an image.
- E. Padding can help preserve spatial dimensions and ensure that edge pixels are processed during convolution.

Question

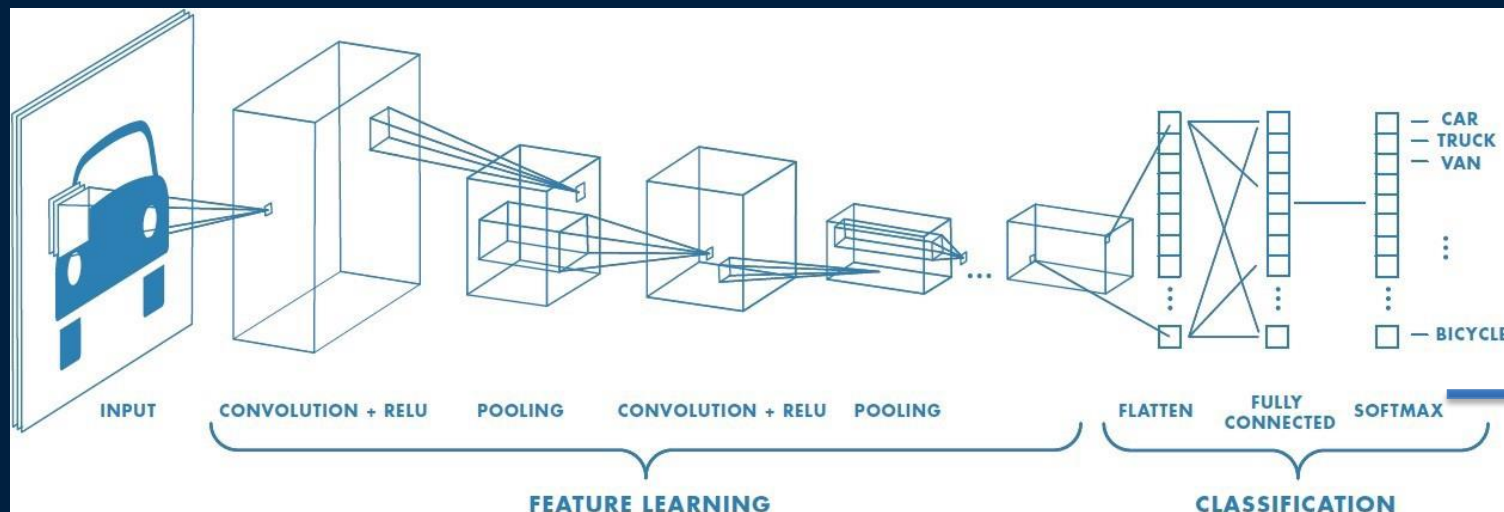
Question: Suppose the input is a 32×32 RGB image, and we use 10 filters of size 5×5 , stride = 1, padding = 0.

What is the output size of feature map?

- A. $32 \times 32 \times 10$
- B. $28 \times 28 \times 10$
- C. $27 \times 27 \times 10$
- D. $28 \times 28 \times 3$
- E. $5 \times 5 \times 10$

CNN Tasks

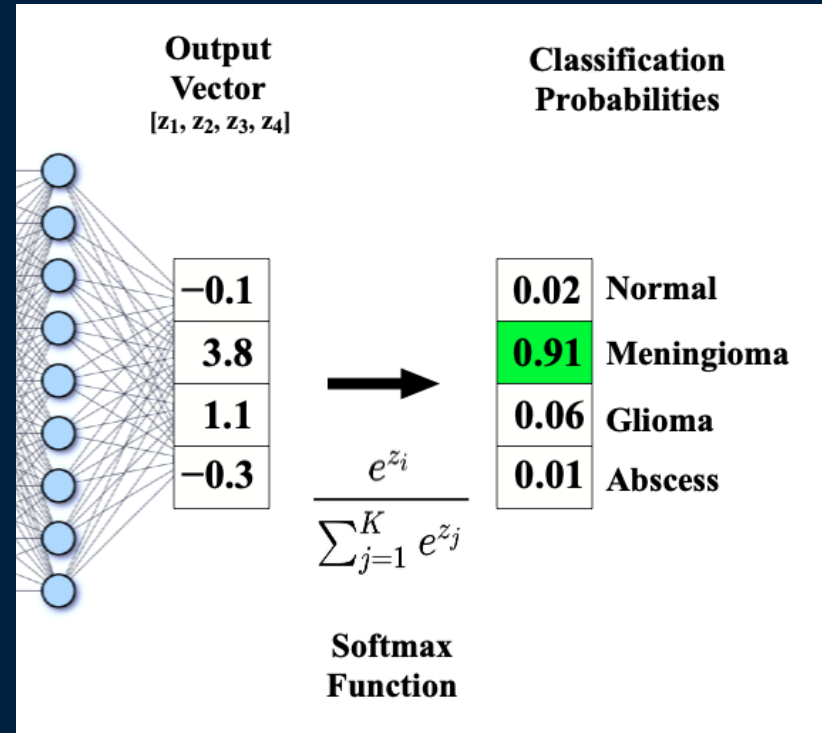
CNNs for Classification: Feature Learning



$$\text{softmax}(y_i) = \frac{e^{y_i}}{\sum_j e^{y_j}}$$

Softmax Function

For classification you need to have **probability distribution** and to have probability distribution you need to have **math to sum to 1**.



CNN in PyTorch



```
import torch
import torch.nn as nn
```

```
def generate_model():
    model = nn.Sequential(
```

```
        # ----- Feature Extraction /first and second conv layers-----
        nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
```

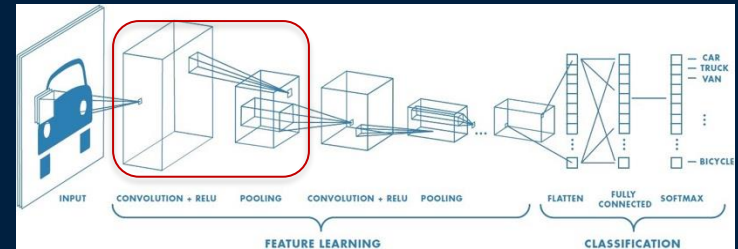
```
        nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),
```

```
        # ----- Classification -----
```

```
        nn.Flatten(),
        nn.Linear(64 * 8 * 8, 1024),    # depends on input image size
        nn.ReLU(),
        nn.Linear(1024, 10)            # number of classes
```

```
)
```

```
return model
```



CNN in PyTorch



```
import torch
import torch.nn as nn

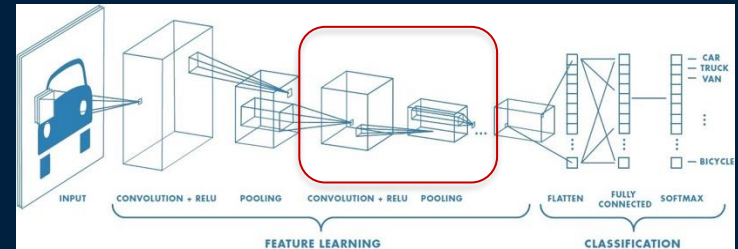
def generate_model():
    model = nn.Sequential(

        # ----- Feature Extraction /first and second conv layers-----
        nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),

        nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),

        # ----- Classification -----
        nn.Flatten(),
        nn.Linear(64 * 8 * 8, 1024),    # depends on input image size
        nn.ReLU(),
        nn.Linear(1024, 10)           # number of classes
    )

    return model
```



CNN in PyTorch



```
import torch
import torch.nn as nn

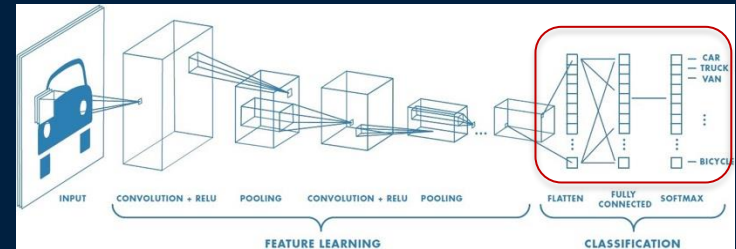
def generate_model():
    model = nn.Sequential(

        # ----- Feature Extraction /first and second conv layers-----
        nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),

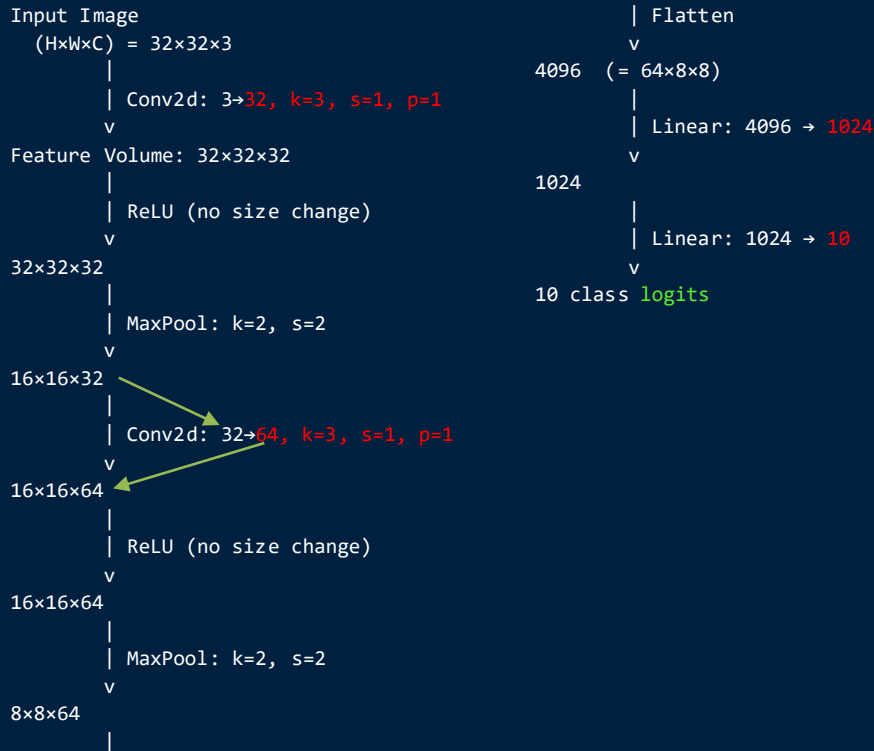
        nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),

        # ----- Classification -----
        nn.Flatten(),
        nn.Linear(64 * 8 * 8, 1024),    # depends on input image size
        nn.ReLU(),
        nn.Linear(1024, 10)           # number of classes
    )

    return model
```



Spatial Dimensions



Red values are
hyperparameters

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - H}{S} \right\rfloor + 1$$

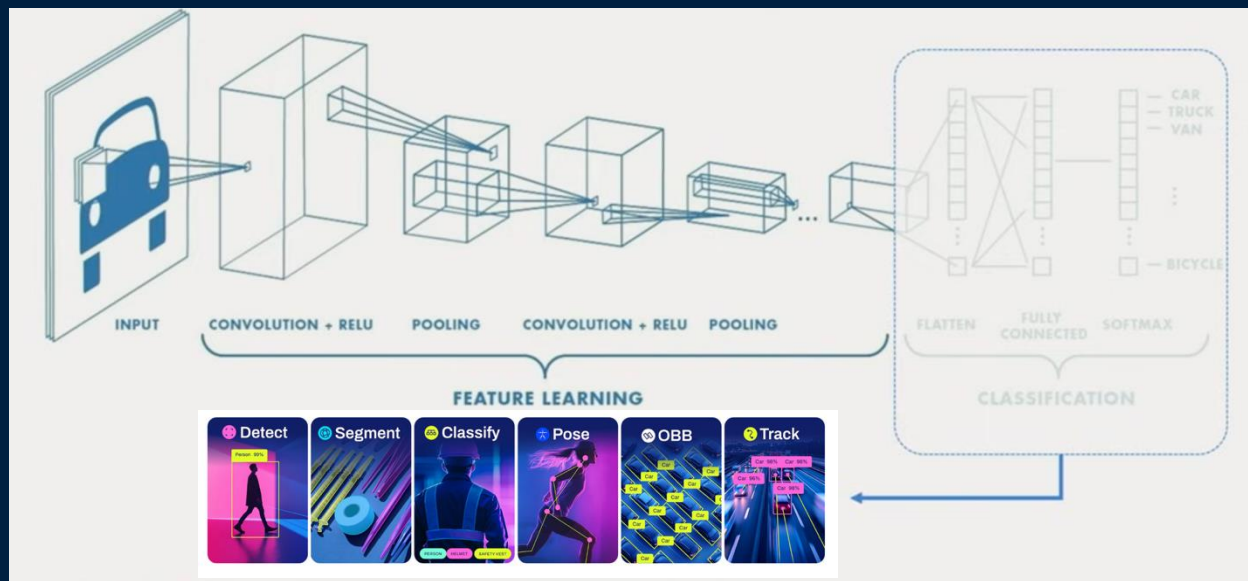
$$W_{out} = \left\lfloor \frac{W_{in} + 2P - W}{S} \right\rfloor + 1$$

$$H_{out} = \left\lfloor \frac{32 + 2(1) - 3}{1} \right\rfloor + 1 = \lfloor 31 \rfloor + 1 = 32$$

$$W_{out} = \left\lfloor \frac{32 + 2(1) - 3}{1} \right\rfloor + 1 = \lfloor 31 \rfloor + 1 = 32$$

An architecture for Many Applications

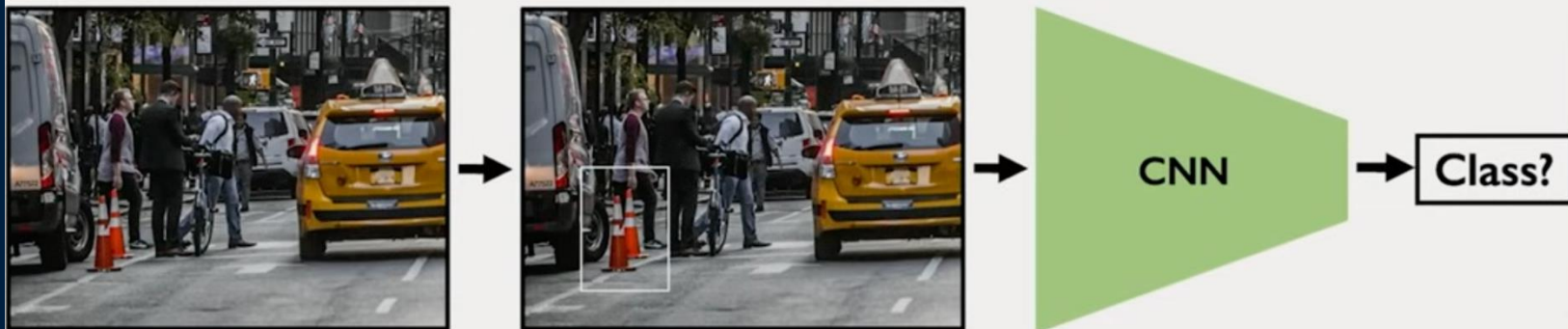
- The **backbone** learns useful visual features such as edges, textures, shapes, and higher-level patterns.
- The **head** takes those extracted features and maps them to the final output, such as class scores.



Object Detection



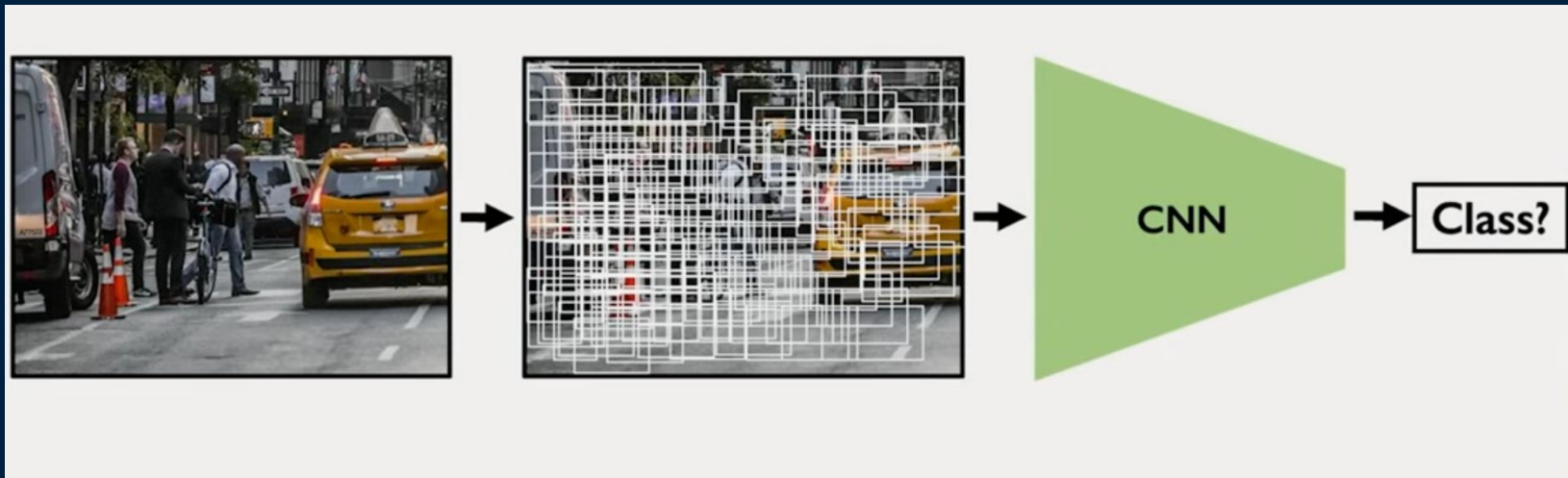
Object Detection Solutions



Object Detection Solutions

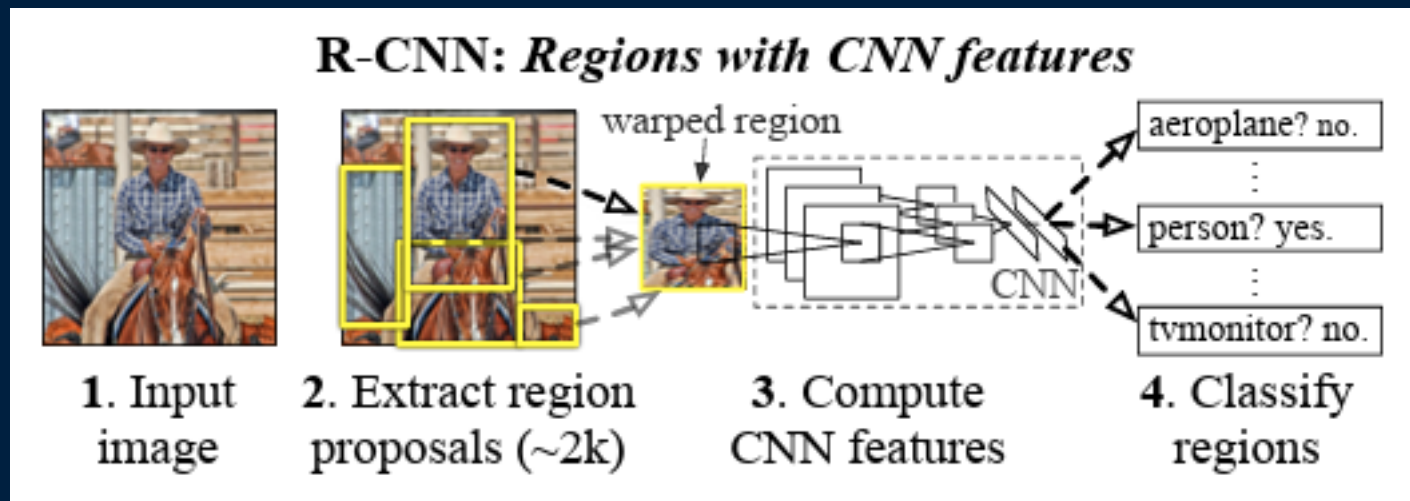


Object Detection Solutions



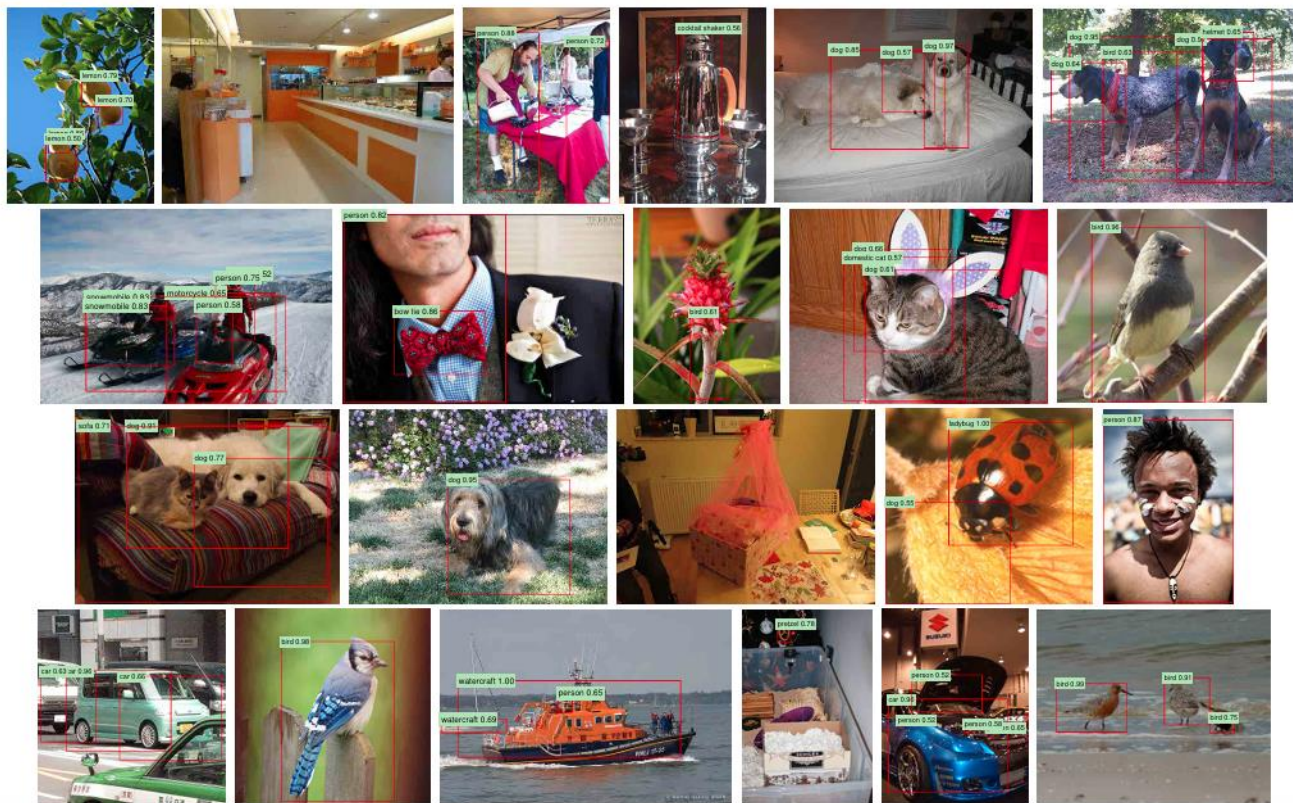
A better method: OD with R-CNNs

R-CNN algorithm: find regions that we have objects. Use CNN to Classify.



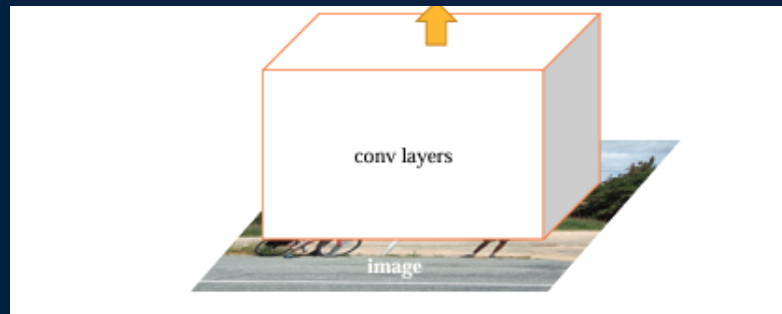
1. Slow and too many regions.
2. Manually defined region proposals.

R-CNN



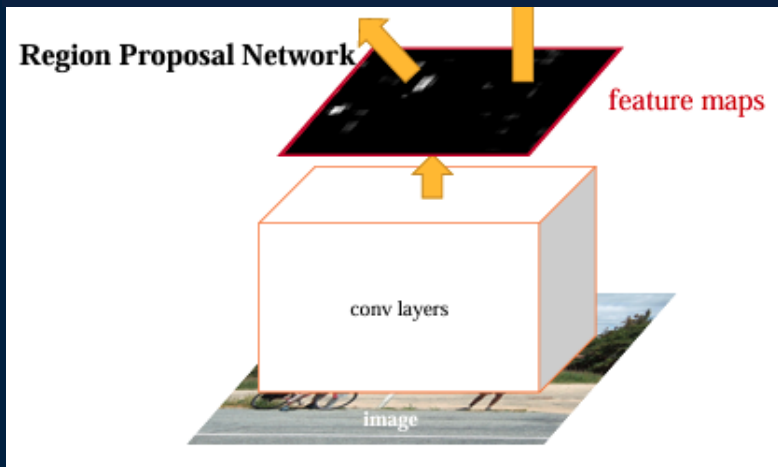
Faster R-CNN

Faster R-CNN replaces heuristic region proposals with a learnable Region Proposal Network, making object detection faster and fully end-to-end.



Faster R-CNN

Faster R-CNN replaces heuristic region proposals with a learnable Region Proposal Network, making object detection faster and fully end-to-end.



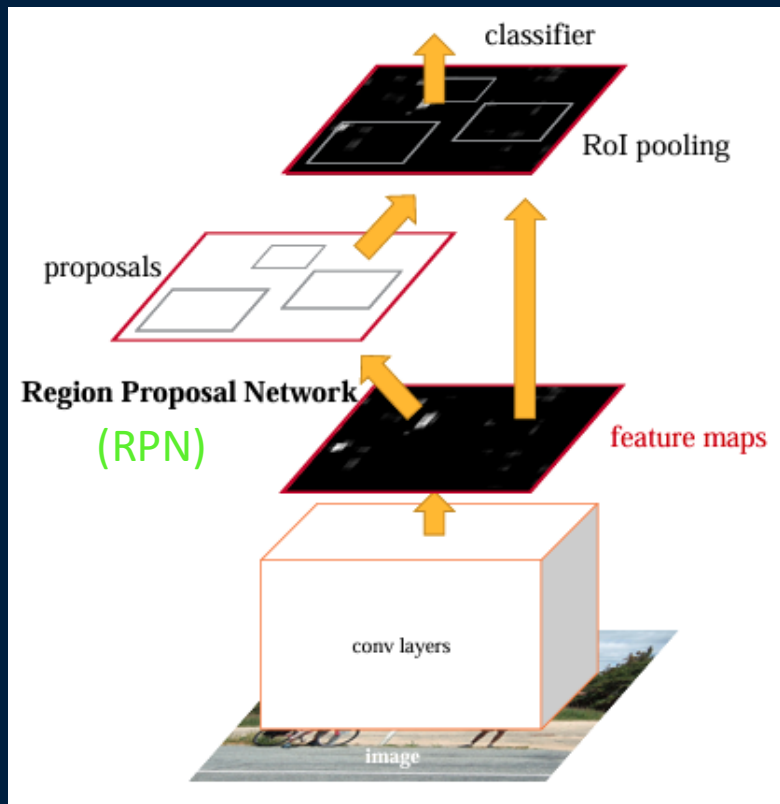
Faster R-CNN

Faster R-CNN: Classifier and RPN use the same feature extraction and this makes the network operates faster.

Faster-RCNN and RCNN it is called a **two-stage detector**

Stage 1: Region Proposal

Stage 2: Classification and Box Refinement



Single-Stage Object Detectors

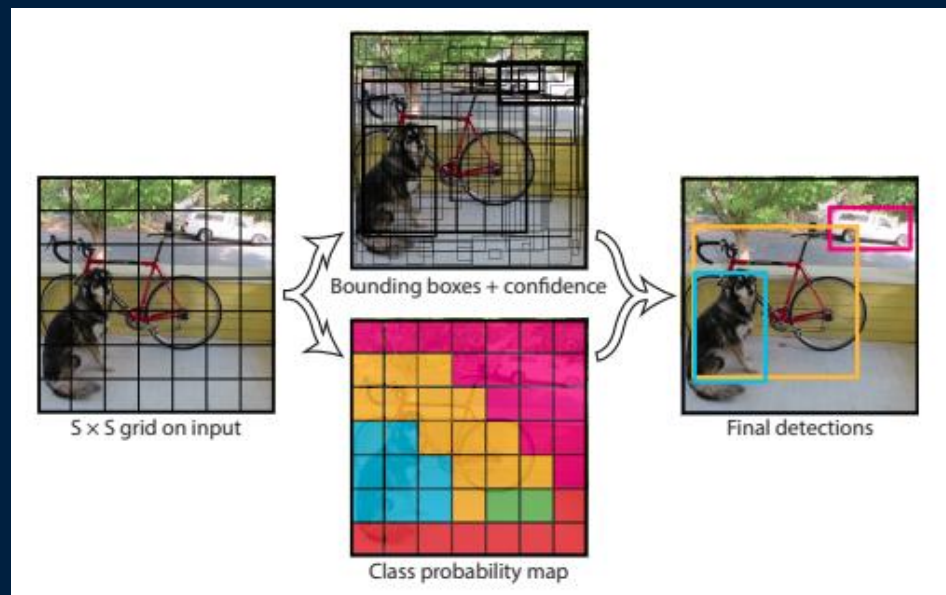
Faster R-CNN still uses two stages — YOLO, SSD, RetinaNet remove region proposals entirely.

- **One-stage detector** (no region proposals)
- **Fully end-to-end**
- **Extremely fast**
- Optimized for **real-time detection**

Yolo: You Only Look Once

1. Resize the image
2. Run convolution network
3. Apply threshold detections

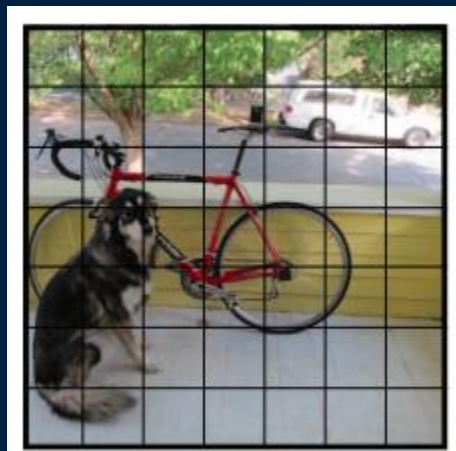
all in one forward pass of a
single neural network.



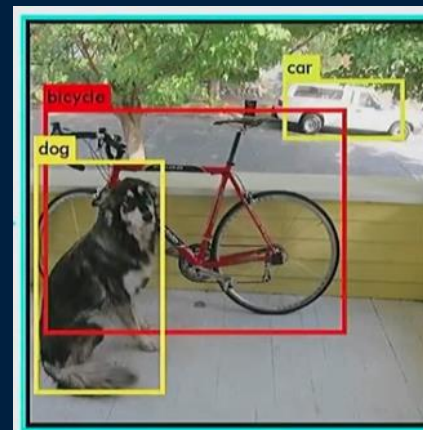
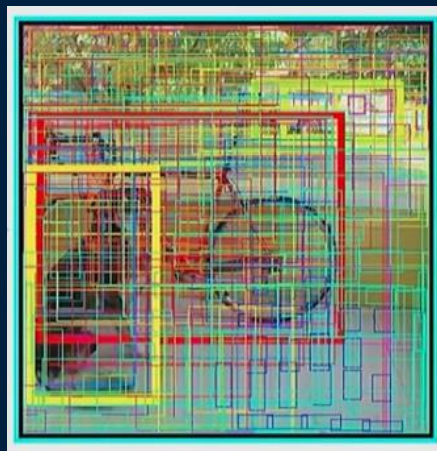
Yolo

Apply a grid. Each cell is responsible to predict:

1. Boxes (x,y,h,w) , B
2. $P(\text{object}) = \text{Confidence}$
3. Class probability



SxS Grid



Apply threshold and none

Non Max Suppression (NMS)

Yolo

Why YOLO Is Fast?

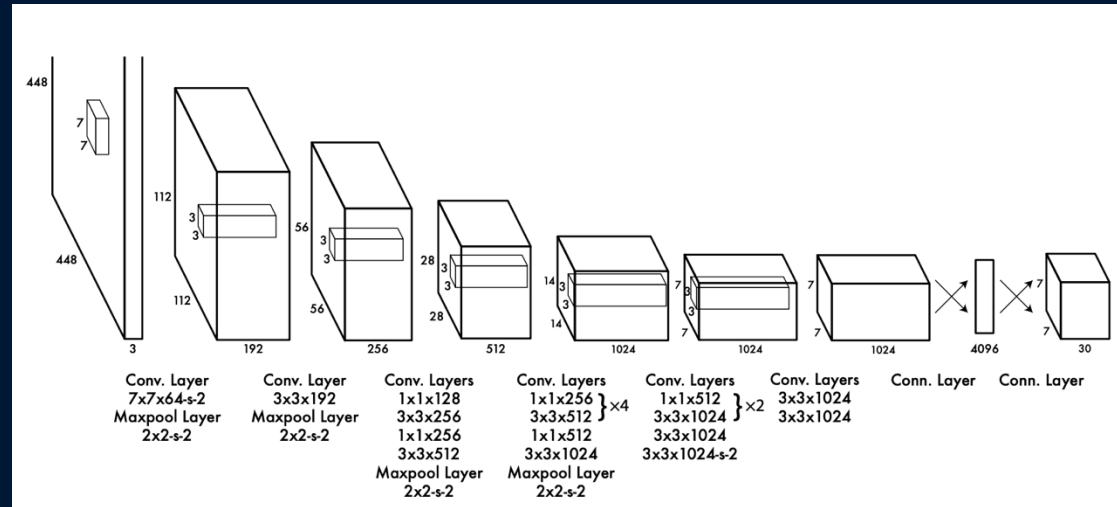
- Single network
- No proposal generation
- No per-region processing
- GPU-friendly dense predictions

Strengths:

- Real-time performance
- Simple pipeline
- End-to-end training
- Widely used in industry (autonomous driving, robotics, surveillance)

Limitations (Early Versions)

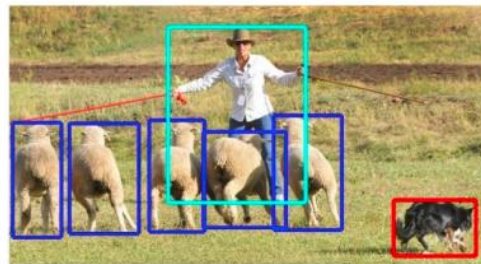
- Struggles with small objects
- Localization less precise than two-stage detectors



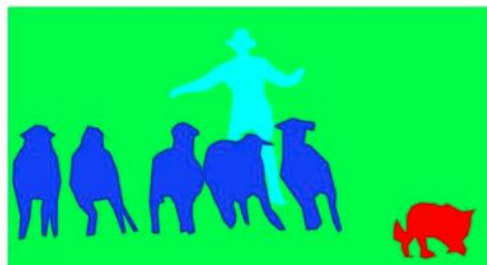
Semantic Segmentation



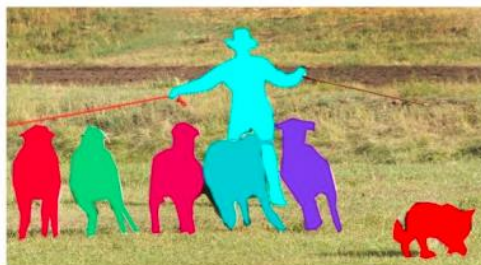
image classification



object detection

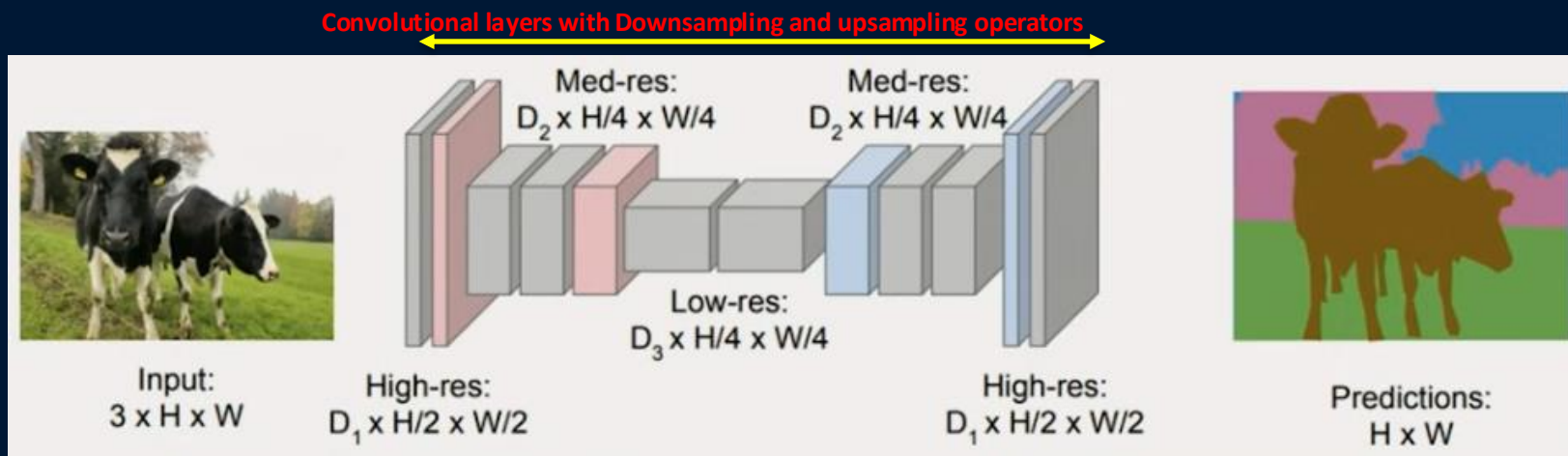


semantic segmentation



instance segmentation

Semantic Segmentation



Objectives

By the end of this lecture, students should be able to:

- Explain the main building blocks of a CNN: convolution, activation, and pooling
- Describe how filters, feature maps, stride, and padding affect spatial dimensions
- Compute the output size of convolution and pooling layers
- Build intuition for how CNNs extract visual features from local regions
- Distinguish between backbone feature extraction and classification head
- Explain how CNNs are used for image classification
- Compare CNN-based object detection approaches: R-CNN, Faster R-CNN, and YOLO
- Describe the difference between two-stage and single-stage object detectors
- Understand the basic idea of semantic segmentation as pixel-level prediction





THE UNIVERSITY OF BRITISH COLUMBIA

